

ML Models for Text: Logistic Regression

What is Logistic Regression?

Logistic Regression is a classification algorithm that predicts the probability of an input belonging to a particular class. Despite the name suggesting regression, it is used for classification tasks. It works by calculating a weighted sum of the input features and passing that sum through a function (the sigmoid function) that squashes the result into a value between 0 and 1, representing the probability of the positive class. A threshold, typically 0.5, is then used to assign the final class label.

For text classification specifically, Logistic Regression is a strong starting point because it works well with the high-dimensional, sparse vectors produced by TF-IDF and it is fast to train even on large datasets like the 50,000 review IMDB dataset.

Training the Model:

I will continue directly from the TF-IDF representation of the IMDB dataset created earlier.

▼ Logistic Regression

```
[6]
✓ 13s import pandas as pd
      from sklearn.feature_extraction.text import TfidfVectorizer
      from sklearn.model_selection import train_test_split

      df = pd.read_csv("IMDB Dataset.csv")

      X_text = df['review']
      y = df['sentiment']

      tfidf = TfidfVectorizer(max_features=5000, stop_words='english')
      X_tfidf = tfidf.fit_transform(X_text)

      X_train, X_test, y_train, y_test = train_test_split(
          X_tfidf, y, test_size=0.2, random_state=42
      )

[7]
✓ 2s from sklearn.linear_model import LogisticRegression

      model = LogisticRegression(max_iter=1000)
      model.fit(X_train, y_train)

      predictions = model.predict(X_test)
```

Evaluation:

Once predictions are made, we need to measure how good they actually are. Rather than relying on a single metric, it is common practice to look at several, since each captures a different aspect of performance.

Accuracy:

Accuracy is the proportion of total predictions that were correct.

Accuracy is easy to interpret but can be misleading on imbalanced datasets. In this case, the IMDB dataset is balanced (25,000 positive, 25,000 negative) so accuracy is a reasonably fair metric here.

Accuracy

```
[8]
✓ 0s from sklearn.metrics import accuracy_score

acc = accuracy_score(y_test, predictions)
print("Accuracy:", acc)

... Accuracy: 0.8896
```

Precision:

Precision measures, out of everything the model predicted as positive, how many were actually positive. It answers the question: "When the model says positive, how often is it right?"

Precision

```
[10]
✓ 0s from sklearn.metrics import precision_score

prec = precision_score(y_test, predictions, pos_label='positive')
print("Precision:", prec)

... Precision: 0.879167469647331
```

Recall

Recall measures, out of everything that was actually positive, how many the model correctly identified. It answers the question: "Out of all the actual positive reviews, how many did the model catch?"

Recall

```
[9]
✓ 0s from sklearn.metrics import recall_score

rec = recall_score(y_test, predictions, pos_label='positive')
print("Recall:", rec)

... Recall: 0.9053383607858703
```

Precision and recall often trade off against each other, increasing one can decrease the other depending on the classification threshold used.

F1 Score:

F1 score is the harmonic mean of precision and recall providing a single number that balances both. It is particularly useful when you want one overall metric rather than tracking precision and recall separately or when the dataset is imbalanced.

▼ F1 Score

```
[11]
✓ Os from sklearn.metrics import f1_score

f1 = f1_score(y_test, predictions, pos_label='positive')
print("F1 Score:", f1)

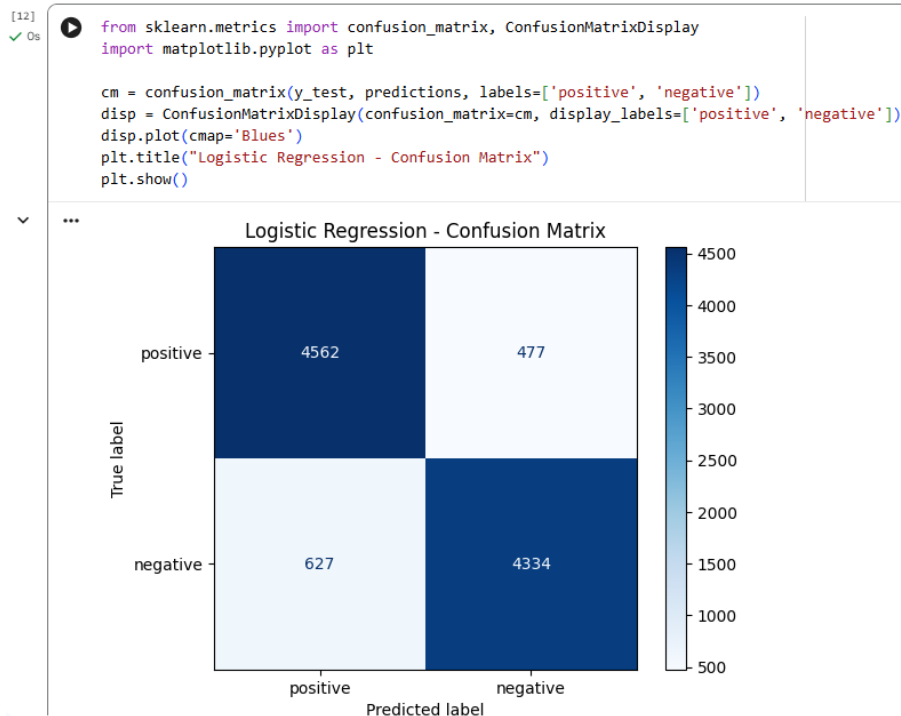
... F1 Score: 0.8920610089949159
```

Confusion Matrix:

A confusion matrix gives a complete breakdown of predictions versus actual labels, showing exactly where the model is making mistakes: true positives, true negatives, false positives, and false negatives.

The confusion matrix makes it easy to see, for example, whether the model is more prone to misclassifying negative reviews as positive or vice versa something a single accuracy number would hide completely.

▼ Confusion Matrix



What is Naive Bayes?

Naive Bayes is a classification algorithm based on Bayes' Theorem which calculates the probability of a class given some observed evidence (in our case, the words in a message). It is called "naive" because it makes a simplifying assumption: it treats every word (feature) as if it were completely independent of every other word, given the class. In reality this is rarely true, the word "free" and the word "prize" often appear together in spam messages, so they're not truly independent but the algorithm works surprisingly well in practice despite this unrealistic assumption.

For text specifically, the most common variant is **Multinomial Naive Bayes** which works directly on word counts or frequencies (like the ones produced by Bag of Words or TF-IDF) and estimates, for each class, how likely each word is to appear. When classifying a new message, it multiplies together the probabilities of each word appearing given each class and picks whichever class comes out with the higher overall probability.

Naive Bayes

```
14]  import pandas as pd
    from sklearn.feature_extraction.text import TfidfVectorizer
    from sklearn.model_selection import train_test_split

    df = pd.read_csv("spam.csv", encoding='latin-1')
    df = df[['v1', 'v2']]
    df.columns = ['label', 'message']

    print(df.shape)
    print(df.head())
    print(df['label'].value_counts())
```

▼

```
... (5572, 2)
   label      message
0  ham  Go until jurong point, crazy.. Available only ...
1  ham                Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3  ham  U dun say so early hor... U c already then say...
4  ham  Nah I don't think he goes to usf, he lives aro...
label
ham      4825
spam     747
Name: count, dtype: int64
```

Accuracy:

Accuracy

```
[18]  from sklearn.metrics import accuracy_score

acc = accuracy_score(y_test, predictions)
print("Accuracy:", acc)
```

▼

```
Accuracy: 0.968609865470852
```

Accuracy measures the overall proportion of correct predictions across both classes: (115 + 965) out of the total 1,115 test messages. A 96.86% accuracy looks very strong but because the dataset is imbalanced (965 ham vs. 150 spam), this number is somewhat inflated by how easy it is to get the abundant ham class right. It doesn't, by itself, tell us how well the model is actually catching spam, for that we need precision and recall.

Precision:

▼ Precision

```
[19] ✓ Os from sklearn.metrics import precision_score  
  
prec = precision_score(y_test, predictions, pos_label='spam')  
print("Precision:", prec)  
  
▼ Precision: 1.0
```

Precision measures, out of everything the model predicted as spam, how much of it was actually spam. Here it is a perfect 1.0 because the confusion matrix shows zero false positives, not a single ham message was incorrectly flagged as spam. In practical terms, this means the model never wrongly blocks a legitimate message which is exactly the behaviour we want from a spam filter since falsely flagging real messages is usually the more damaging kind of error for users.

Recall:

▼ Recall

```
[21] ✓ Os from sklearn.metrics import recall_score  
  
rec = recall_score(y_test, predictions, pos_label='spam')  
print("Recall:", rec)  
  
▼ Recall: 0.7666666666666667
```

Recall measures, out of all the actual spam messages, how many the model successfully caught. Here, 115 out of 150 actual spam messages were correctly identified giving a recall of about 76.7%. The remaining 35 spam messages (23.3%) slipped through undetected as false negatives. This is the weaker side of the model's performance while it never wrongly blocks real messages, it does miss a meaningful portion of actual spam.

F1 Score:

▼ F1 Score

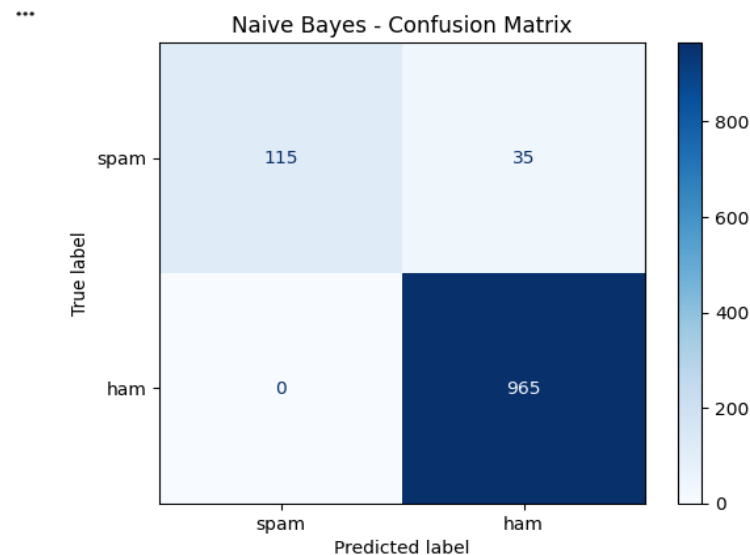
```
[22] ✓ Os from sklearn.metrics import f1_score  
  
f1 = f1_score(y_test, predictions, pos_label='spam')  
print("F1 Score:", f1)  
  
▼ ... F1 Score: 0.8679245283018868
```

F1 score is the harmonic mean of precision and recall, giving a single number that balances both. Since precision (1.0) is much higher than recall (0.767), the F1 score of 0.868 sits between them, pulled down from precision's perfect score because of the lower recall. This reflects the overall trade-off: the model is extremely conservative it only labels something as spam when very confident which eliminates false alarms but costs it some missed detections.

Confusion Matrix:

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, predictions, labels=['spam', 'ham'])
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['spam', 'ham'])
disp.plot(cmap='Blues')
plt.title("Naive Bayes - Confusion Matrix")
plt.show()
```



The confusion matrix shows the model correctly classified 115 out of 150 spam messages and all 965 ham messages, with 35 spam messages misclassified as ham (false negatives) and zero ham messages misclassified as spam (false positives). This confirms that all the model's errors were on the spam side it never wrongly flagged a real message but missed a portion of actual spam.

What is Random Forest?

Random Forest is an ensemble learning algorithm instead of relying on a single model, it builds many individual decision trees and combines their predictions. Each tree is trained on a random subset of the training data and a random subset of features, so each tree ends up slightly different from the others. When making a prediction, every tree "votes" for a class and the forest picks whichever class gets the most votes overall.

This solves a specific weakness of a single decision tree: individual trees tend to over fit easily memorizing quirks of the training data. By averaging across many different, independently-trained trees, Random Forest smooths out those individual mistakes generally leading to better generalization than any single tree on its own.

```
Random Forest

[24] from datasets import load_dataset
✓ 9s
ds = load_dataset("SetFit/bbc-news")
print(ds)

... Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
README.md: 100% ██████████ 880/880 [00:00<00:00, 27.6kB/s]
train.jsonl: 100% ██████████ 2.87M/2.87M [00:00<00:00, 16.5MB/s]
test.jsonl: 100% ██████████ 2.28M/2.28M [00:00<00:00, 14.5MB/s]
Generating train split: 100% ██████████ 1225/1225 [00:00<00:00, 7447.11 examples/s]
Generating test split: 100% ██████████ 1000/1000 [00:00<00:00, 11330.41 examples/s]
DatasetDict({
  train: Dataset({
    features: ['text', 'label', 'label_text'],
    num_rows: 1225
  })
  test: Dataset({
    features: ['text', 'label', 'label_text'],
    num_rows: 1000
  })
})
```

Vectorization

```
[42]
✓ 1s
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_bbc = TfidfVectorizer(max_features=5000, stop_words='english')

X_train_bbc = tfidf_bbc.fit_transform(train_df['text'])
X_test_bbc = tfidf_bbc.transform(test_df['text'])

y_train_bbc = train_df['label_text']
y_test_bbc = test_df['label_text']

[43]
✓ 3s
from sklearn.ensemble import RandomForestClassifier

model_bbc = RandomForestClassifier(n_estimators=200, random_state=42)
model_bbc.fit(X_train_bbc, y_train_bbc)

predictions_bbc = model_bbc.predict(X_test_bbc)
```

Accuracy:

Accuracy

```
[44]
✓ 0s
from sklearn.metrics import accuracy_score

acc = accuracy_score(y_test_bbc, predictions_bbc)
print("Accuracy:", acc)

... Accuracy: 0.946
```

The model correctly classified 94.6% of test articles into one of the five categories. Given that random guessing across five classes would be expected to yield roughly 20% accuracy, this suggests the model is picking up on meaningful vocabulary patterns associated with each category.

Precision:

▼ Precision

```
[45] ✓ 0s from sklearn.metrics import precision_score

      prec = precision_score(y_test_bbc, predictions_bbc, average='weighted')
      print("Precision:", prec)

... Precision: 0.9474094348799033
```

Weighted precision averages precision across all five categories, weighted by the number of articles in each. A value of 94.7% suggests that, on average, when the model assigns a category to an article, it is correct in the majority of cases, with a relatively low rate of articles being incorrectly assigned to a category they don't belong to.

Recall:

▼ Recall

```
[46] ✓ 0s from sklearn.metrics import recall_score

      rec = recall_score(y_test_bbc, predictions_bbc, average='weighted')
      print("Recall:", rec)

... Recall: 0.946
```

Weighted recall of 94.6% suggests that, on average across categories, the model correctly identifies a large majority of articles that actually belong to each class. Since precision and recall are close in value here (94.7% vs. 94.6%), there does not appear to be a strong tendency toward one type of error over the other in this run.

F1 Score:

▼ F1 Score

```
[47] ✓ 0s from sklearn.metrics import f1_score

      f1 = f1_score(y_test_bbc, predictions_bbc, average='weighted')
      print("F1 Score:", f1)

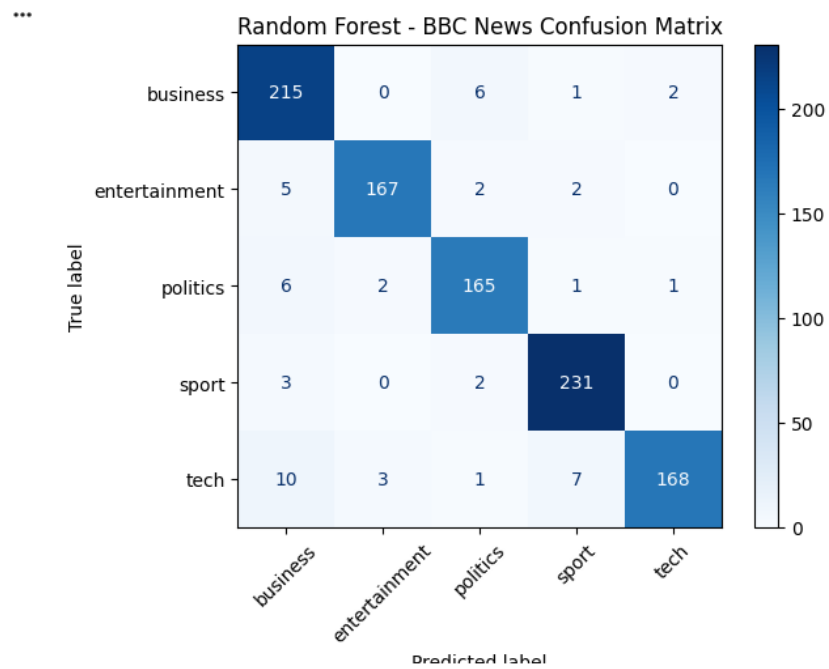
... F1 Score: 0.9459844304052625
```

Because precision and recall are close to each other, the F1 score comes out similarly at 94.6%, which is consistent with a model that does not appear to be trading off one metric heavily for the other in this case.

Confusion Matrix:

```
import matplotlib.pyplot as plt

labels = sorted(y_test_bbc.unique())
cm = confusion_matrix(y_test_bbc, predictions_bbc, labels=labels)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(cmap='Blues', xticks_rotation=45)
plt.title("Random Forest - BBC News Confusion Matrix")
plt.show()
```



What is SVM?

Support Vector Machine is a classification algorithm that works by finding the boundary (called a hyperplane) that best separates data points belonging to different classes. Rather than just finding any boundary that separates the classes, SVM specifically looks for the one that maximizes the margin the distance between the boundary and the closest data points from each class (called support vectors). A larger margin generally means the model is likely to generalize better to new, unseen data.

For text classification, SVM is historically considered one of the strongest traditional (pre-deep-learning) algorithms particularly when working with TF-IDF or Bag of Words features. This is because text data tends to produce very high-dimensional, sparse feature vectors (thousands of columns, mostly zeros) and SVM tends to handle this kind of data well, since it focuses only on the data points closest to the boundary rather than trying to model the entire distribution of the data.

▼ SVM

```
[1]
✓ 21s
from sklearn.datasets import fetch_20newsgroups

categories = ['sci.space', 'rec.sport.baseball', 'talk.politics.guns', 'comp.graphics', 'sci.med']

train_data = fetch_20newsgroups(subset='train', categories=categories, remove=('headers', 'footers', 'quotes'))
test_data = fetch_20newsgroups(subset='test', categories=categories, remove=('headers', 'footers', 'quotes'))

print("Training samples:", len(train_data.data))
print("Test samples:", len(test_data.data))
print("Categories:", train_data.target_names)
```

▼

```
Training samples: 2914
Test samples: 1940
Categories: ['comp.graphics', 'rec.sport.baseball', 'sci.med', 'sci.space', 'talk.politics.guns']
```

Accuracy:

▼ Accuracy

```
[4]
✓ 0s
from sklearn.metrics import accuracy_score

acc = accuracy_score(y_test_svm, predictions_svm)
print("Accuracy:", acc)
```

▼

```
Accuracy: 0.8262886597938144
```

The model correctly classified about 82.6% of test articles into one of the five categories. This is noticeably lower than what was observed with Random Forest on the BBC News dataset (94.6%), which may suggest that the 20 Newsgroups categories used here are harder to distinguish from one another, though this could also reflect differences between the two datasets themselves rather than a direct comparison of the two algorithms.

Precision:

▼ Precision

```
[5] ✓ 0s from sklearn.metrics import precision_score

prec = precision_score(y_test_svm, predictions_svm, average='weighted')
print("Precision:", prec)

... Precision: 0.8280180756722589
```

Weighted precision of 82.8% suggests that, on average across the five categories, a moderate proportion of predictions for each category were correct, though noticeably less consistent than in the BBC News example.

Recall:

▼ Recall

```
[6] ✓ 0s from sklearn.metrics import recall_score

rec = recall_score(y_test_svm, predictions_svm, average='weighted')
print("Recall:", rec)

... Recall: 0.8262886597938144
```

Recall came out nearly identical to precision (82.6% vs. 82.8%) which suggests the model isn't leaning strongly toward one type of error over another in this run.

F1 Score:

▼ F1 Score

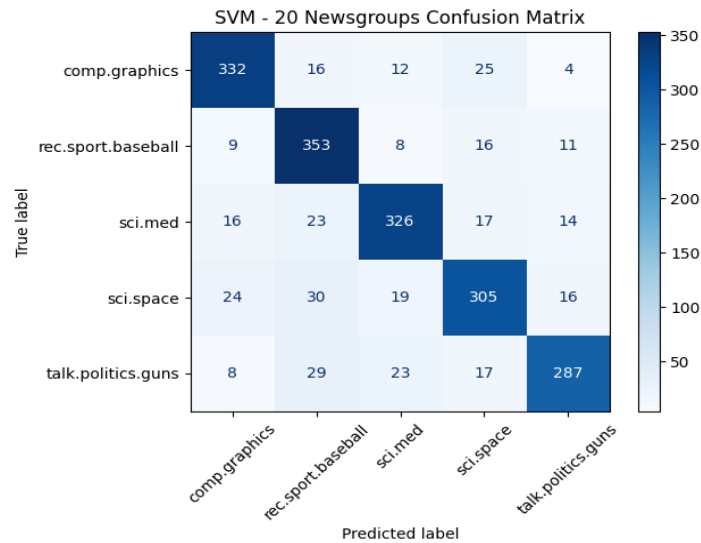
```
[7] ✓ 0s from sklearn.metrics import f1_score

f1 = f1_score(y_test_svm, predictions_svm, average='weighted')
print("F1 Score:", f1)

... F1 Score: 0.8260618325731047
```

Since precision and recall are close in value, the F1 score sits at a similar 82.6%, consistent with a model that appears fairly balanced between the two, even if overall performance is more modest than in the earlier multi-class example.

Confusion Matrix:



The confusion matrix shows most predictions fall along the diagonal, with rec.sport.baseball having the fewest errors and sci.space showing the most confusion. Misclassifications appear spread across categories rather than concentrated in one or two pairs, suggesting general vocabulary overlap.

References:

<https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/>

https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html

<https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/>

<https://medium.com/low-code-for-advanced-data-science/support-vector-machines-svm-an-intuitive-explanation-b084d6238106>